Hash Tables: Separate Chaining vs. Open Addressing

When multiple keys map to the same array index via the hash function, a **collision** occurs. Since a single array slot can only hold one value by default, hash tables use collision resolution strategies. The two primary approaches are **Separate Chaining** and **Open Addressing (Linear Probing)**.

---

1. Separate Chaining

Concept

In separate chaining, each index of the hash table array points to a **linked list** of entries. When a collision occurs, the new key-value pair is simply appended to the linked list at that index. Multiple items with the same hash code coexist in the same bucket.

Step-by-Step Example

Assume a hash table with `capacity = 4` and a hash function `key.length() % capacity`.

1. **`put("cat", "ValueA")`**: Length is 3. $3 \pmod 4 = 3$. Index `[3]` is empty. Place node directly.
2. **`put("dog", "ValueB")`**: Length is 3. $3 \pmod 4 = 3$. Index `[3]` is already occupied ("cat"). **Collision!**
   - Append a new node for `"dog"` to the linked list at index `[3]`. The chain becomes: `[cat] -> [dog]`.
3. **`get("dog")`**: Hash is 3. Traverse the linked list at index `[3]`, checking keys until `"dog"` is found.

---

2. Open Addressing (Linear Probing)

Concept

In open addressing, **all elements are stored directly within the hash table array itself**—no external linked lists. When a collision occurs, the algorithm systematically probes (searches) through alternative slots in the array using a fixed sequence (most commonly **linear probing**, where it checks the next index: `index = (hash + i) % capacity`) until an empty slot is found.

Step-by-Step Example

Assume `capacity = 4`, hash function `key.length() % capacity`.

1. **`put("cat", "ValueA")`**: Length 3 $\rightarrow$ Index `[3]`. Empty, place node.
2. **`put("dog", "ValueB")`**: Length 3 $\rightarrow$ Index `[3]`. Occupied!
   - **Linear Probe:** Check index `(3 + 1) % 4 = 0`. Index `[0]` is empty. Place `"dog"` at index `[0]`.
3. **`get("dog")`**: Hash is 3. Index `[3]` holds `"cat"` (not `"dog"`). Probe next index `[0]`, find `"dog"`, and return its value.

---

3. Time Complexity and Load Factor Analysis

The performance of both strategies depends heavily on the **Load Factor (α)**, defined as:

$$\alpha = \frac{N}{M} = \frac{\text{Number of elements}}{\text{Table capacity}}$$

Complexity Comparison Table

Operation	Strategy	Best Case	Average Case ($\alpha \ll 1$)	High Load Factor ($\alpha \approx 0.7 - 0.9$)	Worst Case ($\alpha \to 1$ or all collide)
Search (Hit / Miss)	Separate Chaining	$\mathcal{O}(1)$	$\mathcal{O}(1 + \alpha)$	$\mathcal{O}(1 + \alpha)$	$\mathcal{O}(N)$
	Open Addressing (Linear Probing)	$\mathcal{O}(1)$	$\mathcal{O}\left(1 + \frac{1}{1-\alpha}\right)$	$\mathcal{O}\left(1 + \frac{1}{(1-\alpha)^2}\right)$	$\mathcal{O}(N)$
Insertion	Separate Chaining	$\mathcal{O}(1)$	$\mathcal{O}(1)$ or $\mathcal{O}(1 + \alpha)$	$\mathcal{O}(1 + \alpha)$	$\mathcal{O}(N)$
	Open Addressing	$\mathcal{O}(1)$	$\mathcal{O}\left(1 + \frac{1}{1-\alpha}\right)$	Degrades rapidly as table fills	$\mathcal{O}(N)$
Deletion	Separate Chaining	$\mathcal{O}(1)$	$\mathcal{O}(1 + \alpha)$	$\mathcal{O}(1 + \alpha)$	$\mathcal{O}(N)$
	Open Addressing	$\mathcal{O}(1)$	$\mathcal{O}\left(1 + \frac{1}{1-\alpha}\right)$	Requires probe sequence rebuild	$\mathcal{O}(N)$

4. Java Implementations with Dynamic Resizing

When the load factor exceeds 0.75, both tables automatically resize and rehash to preserve $\mathcal{O}(1)$ performance.

Updated `Node` Class

```
package hashtable;

public class Node {
    String key;
    String value;
    Node next; // Used for Separate Chaining

    public Node() {}

    public Node(String key, String value) {
        this.key = key;
        this.value = value;
        this.next = null;
```

```
    }
}
```

Separate Chaining HashTable (With Dynamic Resizing)

```
package hashtable;

public class SeparateChainingHashTable {
    Node[] table;
    int capacity;
    int size;

    public SeparateChainingHashTable(int capacity) {
        table = new Node[capacity];
        this.capacity = capacity;
        this.size = 0;
    }

    public int hashkey(String key) {
        return key.length() % this.capacity;
    }

    private void resize() {
        int oldCapacity = this.capacity;
        Node[] oldTable = this.table;

        this.capacity = oldCapacity * 2;
        this.table = new Node[this.capacity];
        this.size = 0;

        for (int i = 0; i < oldCapacity; i++) {
            Node current = oldTable[i];
            while (current != null) {
                put(current.key, current.value);
                current = current.next;
            }
        }
        System.out.println("Resized SeparateChainingHashTable. New
capacity: " + this.capacity);
    }

    public void put(String key, String value) {
        if ((float)(this.size + 1) / this.capacity > 0.75) {
            resize();
        }

        int hash = hashkey(key);
        Node newNode = new Node(key, value);

        if (this.table[hash] == null) {
            this.table[hash] = newNode;
```

```
        } else {
            Node current = this.table[hash];
            while (true) {
                if (current.key.equals(key)) {
                    current.value = value;
                    return;
                }
                if (current.next == null) {
                    current.next = newNode;
                    break;
                }
                current = current.next;
            }
        }
        this.size++;
    }

    public String get(String key) {
        int hash = hashkey(key);
        Node current = this.table[hash];
        while (current != null) {
            if (current.key.equals(key)) {
                return current.value;
            }
            current = current.next;
        }
        return null;
    }

    public void show() {
        System.out.printf("\nLoad Factor: %3.2f (Size: %d, Capacity:
%d)\n",
            (float)this.size / this.capacity, this.size, this.capacity);
        System.out.printf("--------------------------\n");
        for (int index = 0; index < this.capacity; index++) {
            System.out.printf("[%d]: ", index);
            Node current = this.table[index];
            if (current == null) {
                System.out.printf("NULL\n");
            } else {
                while (current != null) {
                    System.out.printf("[Key: %s, Val: %s] -> ",
current.key, current.value);
                    current = current.next;
                }
                System.out.printf("null\n");
            }
        }
    }
}
```

Open Addressing HashTable (With Linear Probing and Dynamic Resizing)

```
package hashtable;

public class OpenAddressingHashTable {
    Node[] table;
    int capacity;
    int size;

    public OpenAddressingHashTable(int capacity) {
        table = new Node[capacity];
        this.capacity = capacity;
        this.size = 0;
    }

    public int hashkey(String key) {
        return key.length() % this.capacity;
    }

    private void resize() {
        int oldCapacity = this.capacity;
        Node[] oldTable = this.table;

        this.capacity = oldCapacity * 2;
        this.table = new Node[this.capacity];
        this.size = 0;

        for (int i = 0; i < oldCapacity; i++) {
            if (oldTable[i] != null) {
                put(oldTable[i].key, oldTable[i].value);
            }
        }
        System.out.println("Resized OpenAddressingHashTable. New capacity:
" + this.capacity);
    }

    public void put(String key, String value) {
        if ((float)(this.size + 1) / this.capacity > 0.75) {
            resize();
        }

        int hash = hashkey(key);

        while (this.table[hash] != null) {
            if (this.table[hash].key.equals(key)) {
                this.table[hash].value = value;
                return;
            }
            hash = (hash + 1) % this.capacity;
        }

        this.table[hash] = new Node(key, value);
        this.size++;
    }
```

```
    public String get(String key) {
        int hash = hashkey(key);
        int originalHash = hash;

        while (this.table[hash] != null) {
            if (this.table[hash].key.equals(key)) {
                return this.table[hash].value;
            }
            hash = (hash + 1) % this.capacity;
            if (hash == originalHash) break;
        }
        return null;
    }

    public void show() {
        System.out.printf("\nLoad Factor: %3.2f (Size: %d, Capacity:
%d)\n",
            (float)this.size / this.capacity, this.size, this.capacity);
        System.out.printf("---------------------------\n");
        for (int index = 0; index < this.capacity; index++) {
            if (this.table[index] != null) {
                System.out.printf("[%d]: Key: %s Value: %s\n", index,
table[index].key, table[index].value);
            } else {
                System.out.printf("[%d]: NULL\n", index);
            }
        }
    }
}
```